API SECURITY ASSESSMENT REPORT

Target System: OWASP crAPI (Completely Ridiculous API)
Deployment Environment: Docker (localhost:8888)

Author	Samson Budigila
Intership	Future interns
Institution	Institute of Finance Management (IFM), Dar es Salaam
Programme	BSc Cybersecurity
Assessment Type	API Security Assessment / Penetration Testing
Date	April 2026
Classification	Confidential Academic Use Only

TOOLS USED IN THE ASSESSMENT

TOOL	USAGE
Postman	Testing the vulnerabilities
Browser dev tools(network,console)	Register/Login and watching Network tab to see correct requests & OTPs
Docker	Running crAPI

1. Executive Summary

This report presents the findings of an API security assessment conducted on the OWASP crAPI (Completely Ridiculous API) platform, a deliberately vulnerable API designed for security training purposes. The assessment was performed in a controlled Docker environment hosted on localhost:8888, using industry-standard tools including Postman and browser developer tools.

A total of seven (7) vulnerabilities were identified spanning a range of severity levels. These include critical issues such as Broken Object Level Authorization (BOLA), Mass Assignment, and Weak Token handling, as well as high-severity concerns including Missing Rate Limiting, Exposed Information Disclosure, and API Endpoint Discovery. Each vulnerability was manually tested and documented with supporting evidence screenshots.

The findings align with well-known OWASP API Security Top 10 categories, demonstrating real-world attack vectors that could lead to unauthorized data access, privilege escalation, and system abuse. This report provides detailed risk classifications, technical evidence, and actionable remediation recommendations for each identified vulnerability.

Vulnerability Summary:

#	Vulnerability	OWASP Category	Severity	Status
1	API Endpoint Discovery	API9:2023	**MEDIUM**	Confirmed
2	Broken Object Level Authorization (BOLA)	API1:2023	**HIGH**	Confirmed
3	Broken Authorization (Data Access)	API1:2023	**HIGH**	Confirmed
4	Excessive Data Exposure / Info Disclosure	API3:2023	**HIGH**	Confirmed
5	Weak Tokens — JWT & OTP Brute Force	API2:2023	**CRITICAL**	Confirmed
6	Missing Rate Limiting	API4:2023	**HIGH**	Confirmed
7	Mass Assignment / Input Validation Failure	API6:2023	**CRITICAL**	Confirmed

TABLE OF CONTENT

Executive summary 2
Vulnerability summary 2
Introduction 4
Application overview 4
Scope 5
Methodology 6
Vulnerabilities found 6
Api endpoint discovery 6
Broken object level authorization 8
Broken authorization 10
Excessive data exposure 11
Weak JWT tokens 13
Missing rate limiting 14
Mass assignment 16
Conclusion 17

2. Introduction

This assessment was conducted on the OWASP crAPI (Completely Ridiculous API) an intentionally vulnerable API application developed by OWASP specifically for security awareness and training. The platform simulates a vehicle management web application and exposes a range of real-world API vulnerabilities commonly found in production systems. The crAPI application was deployed locally using Docker and accessed at http://localhost:8888.

The primary objective of this assessment is to identify, analyze, and document security vulnerabilities present in the crAPI API surface. Each finding is evaluated against the OWASP API Security Top 10 (2023) framework, classified by severity using a standardized risk scale, and accompanied by practical remediation guidance.

2.1 Application Overview

The following screenshots show the crAPI web application interface as observed during the assessment. The application includes a sign-in page and a sign-up page, both of which interact with the underlying API endpoints that were tested.

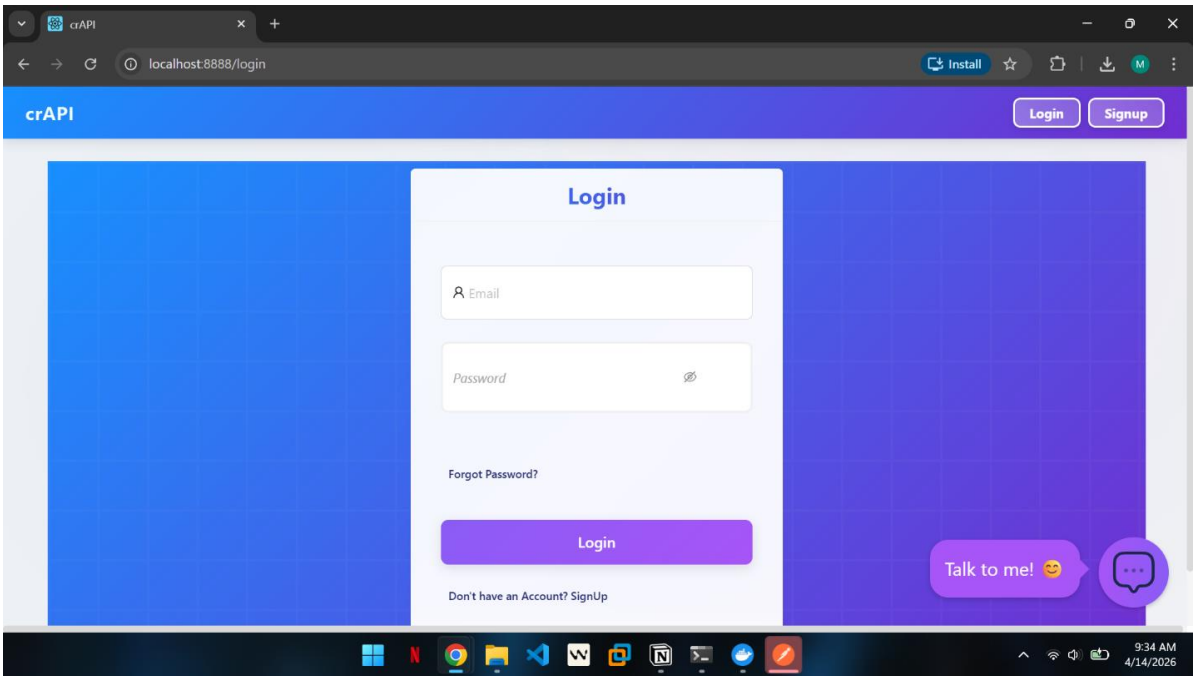

Figure 1: crAPI Sign-In Page

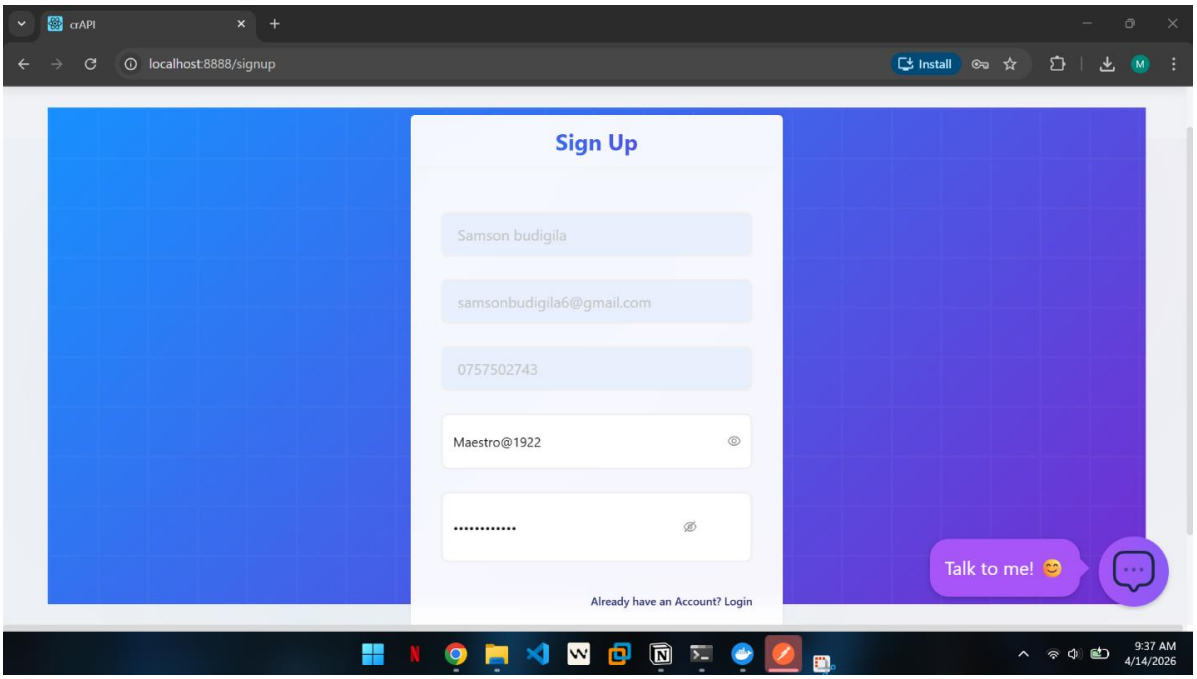

Figure 2: crAPI Sign-Up Page

3. Scope

The scope of this security assessment was limited to the OWASP crAPI application running in a local Docker environment. The assessment was strictly black-box and grey-box in nature, simulating the perspective of an external attacker with limited prior knowledge of the system internals.

Parameter	Details
Target Application	OWASP crAPI
Environment	Docker Container (localhost:8888)
Assessment Type	Manual API Security Assessment
Methodology	OWASP API Security Testing Guide
In Scope	All API endpoints exposed by crAPI including identity, community, vehicle, and shop APIs
Out of Scope	Network infrastructure, operating system, and third-party dependencies
Tools Used	Postman, Browser Developer Tools (Chrome/Firefox)
Testing Period	April 2026

4. Methodology

The assessment followed a structured manual testing approach guided by the OWASP API Security Testing Guide. The process was divided into the following phases:

- **Reconnaissance & Endpoint Discovery**
  Identifying all exposed API endpoints by observing network traffic, analyzing API responses.

- **Authentication & Authorization Testing**
  Testing for broken authentication mechanisms, weak token generation, missing access controls, and privilege escalation vectors.

- **Input Validation Testing**
  Submitting unexpected and unauthorized parameters to identify mass assignment, injection, or improper input handling vulnerabilities.

- **Rate Limiting & Resource Abuse**
  Sending repeated requests to identify missing rate limits and brute force protections.

- **Information Disclosure Analysis**
  Reviewing API responses for sensitive data leakage including internal paths, tokens, and system metadata.

- **Documentation & Reporting**
  Recording all findings with screenshots, OWASP classification, severity scoring, and remediation guidance.

5. Vulnerabilities Found

The following sections detail each vulnerability identified during the assessment. Each finding includes the affected endpoint, a description of the vulnerability, its potential impact, supporting evidence, and recommended remediation steps.

5.1. API Endpoint Discovery

Severity:

MEDIUM

During the reconnaissance phase, I was able to enumerate multiple internal API endpoints by passively observing browser network requests. The crAPI application makes API calls to backend endpoints that are fully visible in the browser's Developer Tools Network tab, including endpoints for product listings and order history.

Vulnerability	API Endpoint Discovery
Endpoint	GET /workshop/api/shop/products | GET /workshop/api/shop/orders

Impact	Exposing internal API endpoints allows attackers to map the entire attack surface of the application, facilitating further exploitation of other vulnerabilities such as BOLA, injection attacks, and data extraction.
Evidence	API endpoints were discovered by observing network traffic in the browser Developer Tools. Endpoints for Products and Orders were visible in plain text, revealing the full API structure without authentication prompts on some routes.

Remediation Steps:

- Implement API gateway rate limiting and endpoint monitoring to detect automated enumeration attempts.
- Ensure all sensitive endpoints require proper authentication before returning data.
- Avoid exposing verbose API paths that reveal internal object names or data structures.

Evidence Screenshots:

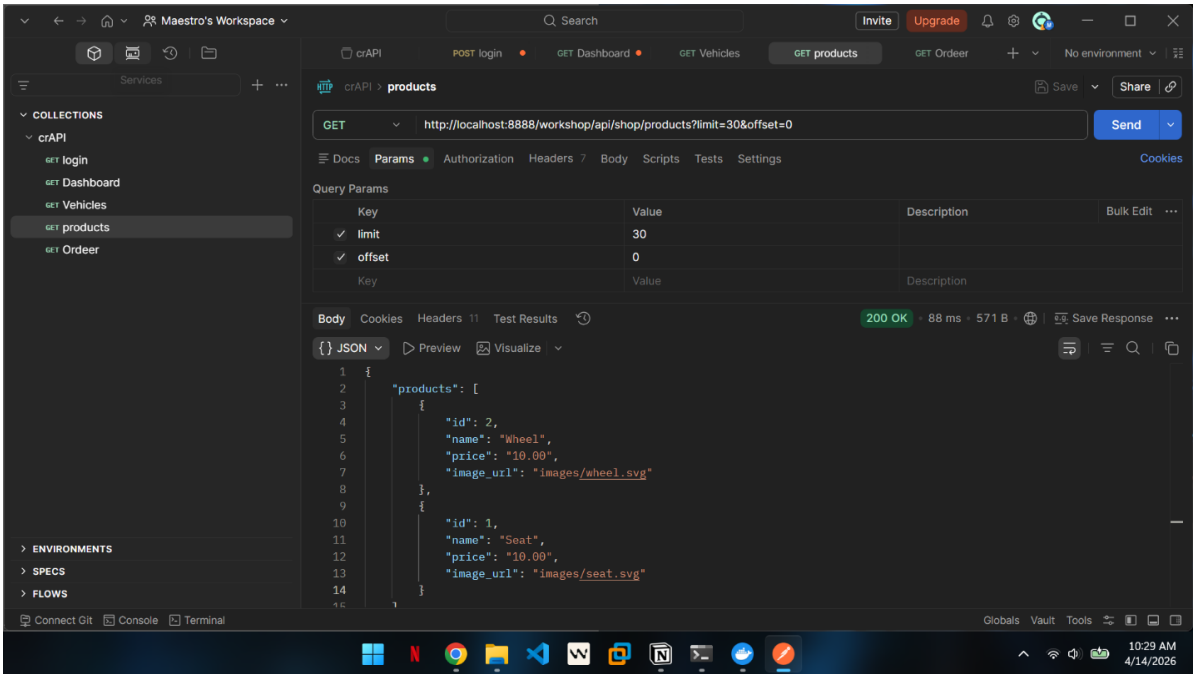

Figure: API Endpoint Discovery — Evidence 1

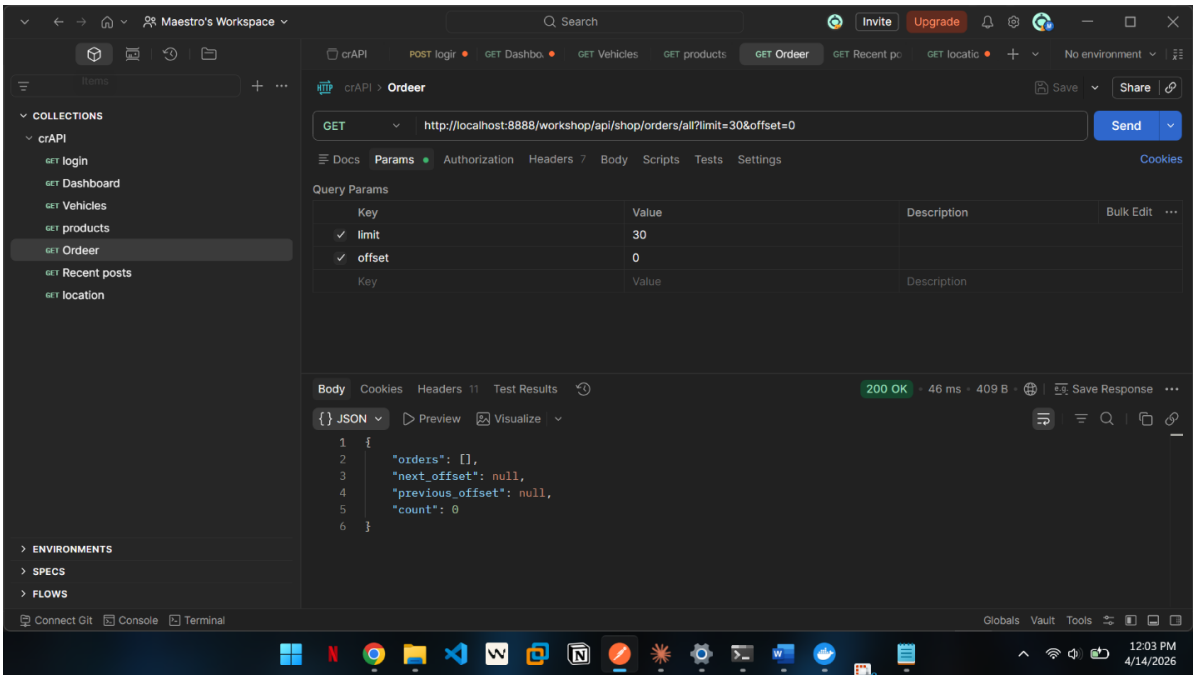

Figure: API Endpoint Discovery — Evidence 2

5.2. Broken Object Level Authorization (BOLA)

Severity:
HIGH

Broken Object Level Authorization (BOLA), also known as Insecure Direct Object Reference (IDOR), is the most prevalent and dangerous API vulnerability as ranked by OWASP. The API fails to validate whether the authenticated user is authorized to access the specific object being requested. By simply changing the vehicleId parameter in the request URL, I was able to retrieve vehicle details belonging to a completely different user account.

Vulnerability	Broken Object Level Authorization (BOLA)
Endpoint	GET /workshop/api/vehicles/{vehicleId}/location
Impact	An authenticated attacker can access vehicle data belonging to other users by substituting their own vehicleId with that of another user. This constitutes a direct violation of horizontal access control and can expose sensitive location and vehicle information.
Evidence	I retrieved their vehicleId from the application, then observed another user's vehicleId through the browser Network tab. By substituting the foreign vehicleId in a Postman request, the API returned full vehicle details for the other user responding with 200 OK.

Remediation Steps:

- Implement server-side object-level authorization checks which verify that the requesting user owns the resource being accessed before returning data.
- Log and alert on access attempts where a user requests objects not associated with their account.

Evidence Screenshots:

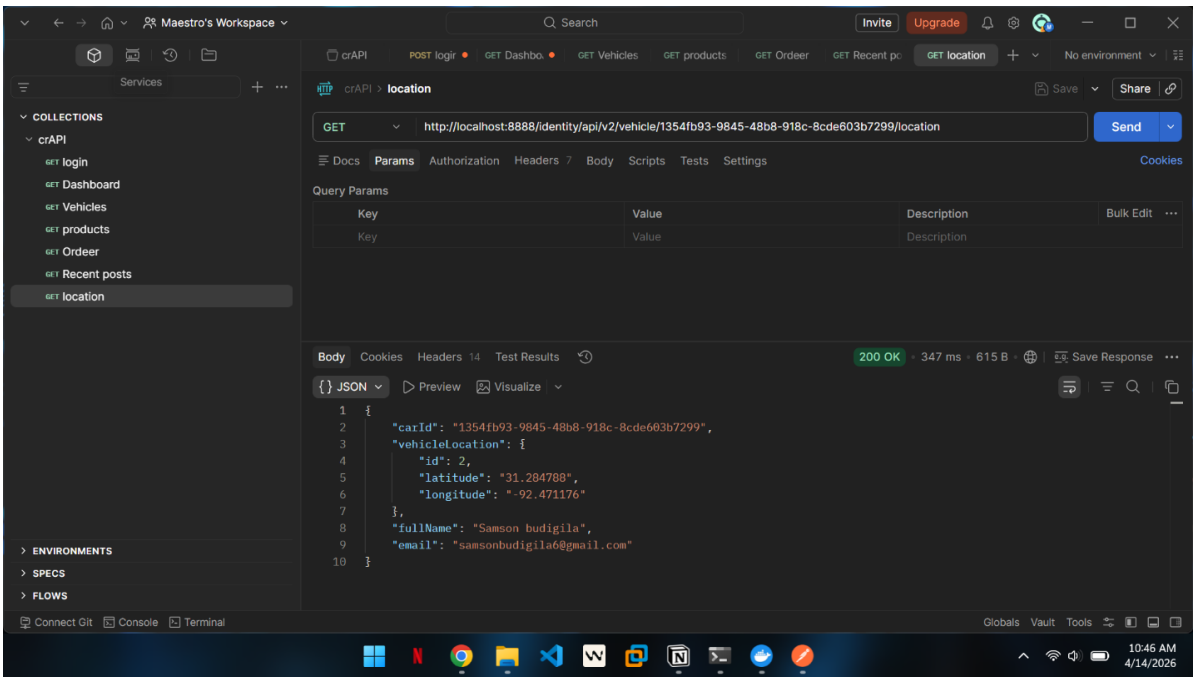

Figure: Broken Object Level Authorization (BOLA) — Evidence 1

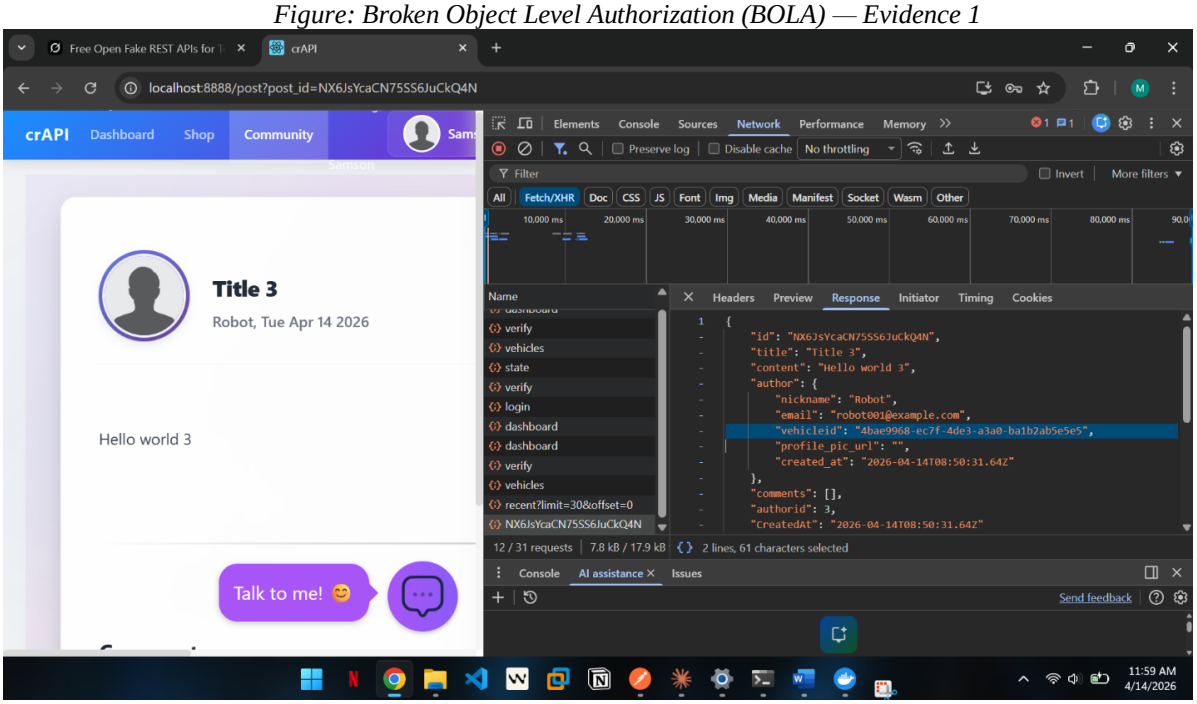

Figure: Broken Object Level Authorization (BOLA) obtaining another user vehichle ID Evidence 2

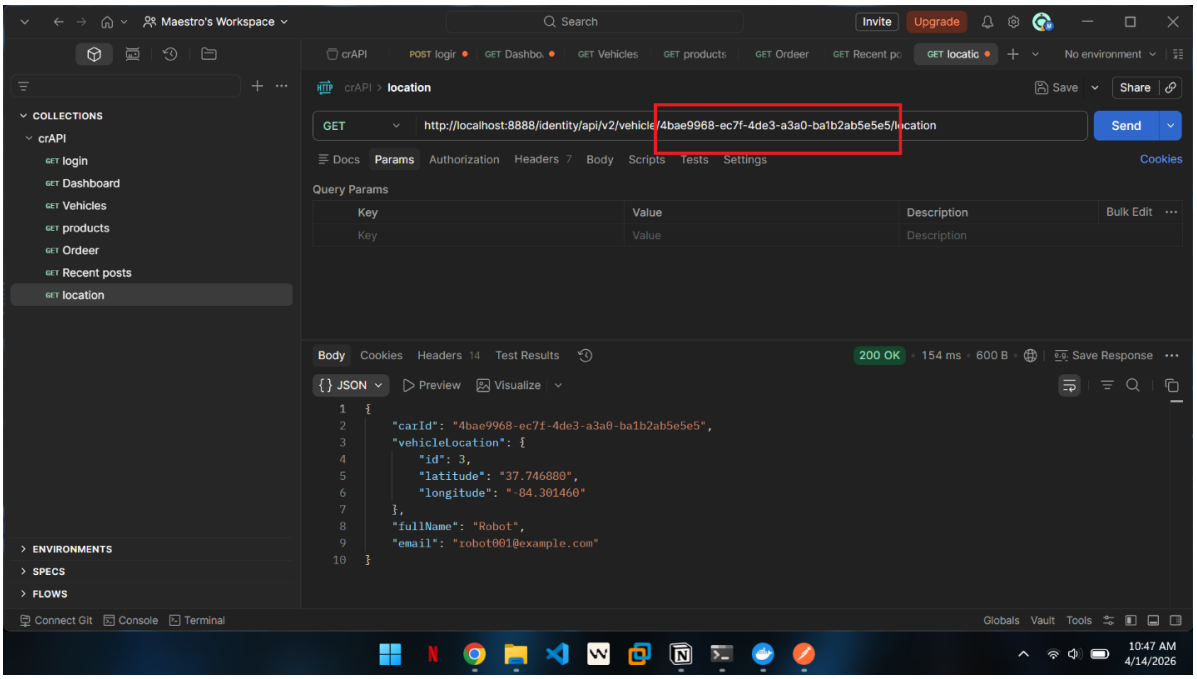

Figure: Broken Object Level Authorization (BOLA) — Evidence 3

5.3. Broken Authorization (Accessing Other Users' Data)

Severity:

HIGH

This vulnerability specifically demonstrates that the authorization failure extends to general car data retrieval endpoints. The lack of ownership enforcement means any authenticated user can retrieve any other user's vehicle information by simply knowing or guessing their car ID.

Vulnerability	Broken Authorization, Accessing Other Users' Data
Endpoint	GET /workshop/api/vehicles/{carId}
Impact	Attackers can access personal data of any user in the system simply by iterating over known or guessable car IDs. This leads to mass data harvesting and potential identity theft if sensitive PII is returned in the response.
Evidence	By changing the Car ID value in the API request, i was able to retrieve vehicle details associated with a different user account. The server returned a 200 OK response with the target user's vehicle data, confirming the absence of ownership verification.

Remediation Steps:

- Enforce strict access control checks at the API layer were every data access request must validate that the authenticated session owner is the resource owner.
- Avoid using sequential or predictable identifiers for objects that contain sensitive information.
- Return 403 Forbidden (not 404) when a user attempts to access a resource they do not own, to prevent enumeration.

Evidence Screenshots:

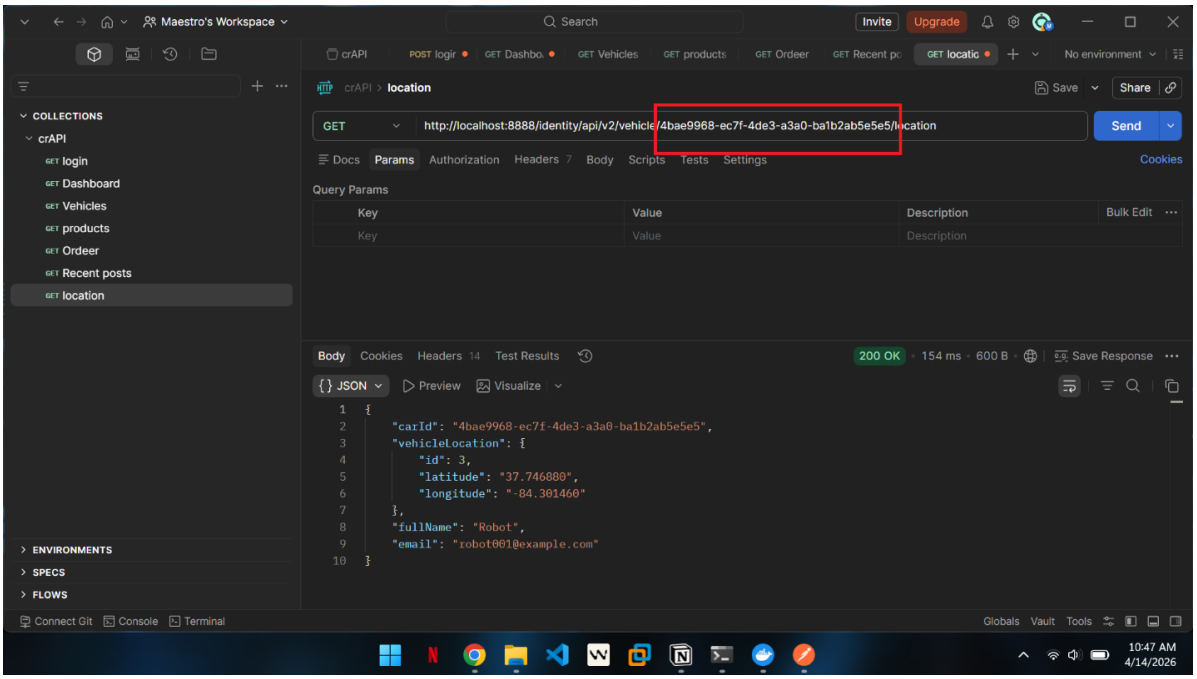

Figure: Broken Authorization — Accessing Other Users' Data — Evidence 1

5.4. Excessive Data Exposure / Information Disclosure

Severity:

HIGH

Excessive Data Exposure occurs when an API endpoint returns more information than necessary, relying on the client application to filter what is displayed. In crAPI, the community posts endpoint reveals user data that is not rendered in the frontend interface but is fully accessible in the raw API response.

Vulnerability	Excessive Data Exposure / Information Disclosure
Endpoint	GET /community/api/v2/community/posts/recent?limit=30&offset=0
Impact	The API returns more data than is necessary for the client's intended function. Sensitive fields such as user identifiers, email addresses,

	vehicle IDs, and internal system metadata may be included in responses, enabling attackers to collect PII and escalate other attacks.
Evidence	A GET request to the community posts endpoint returned a verbose JSON response containing user data beyond what is displayed on the frontend. Fields visible in the raw API response were not shown in the UI, indicating that filtering is done client-side rather than at the API/server level.

Remediation Steps:

- Apply the principle of data minimization, return only the fields that the client application actively needs.
- Implement server-side response filtering rather than relying on the frontend to hide sensitive fields.
- Regularly audit API responses for unintended data exposure using automated tools.

Evidence Screenshots:

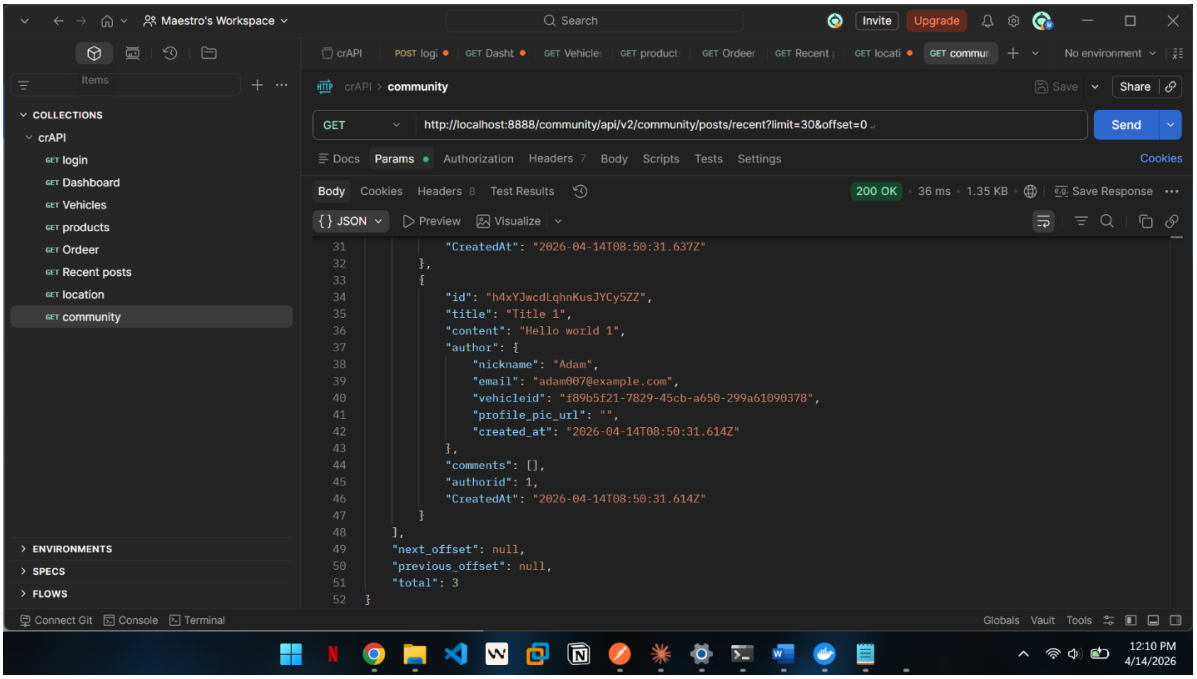

Figure: Excessive Data Exposure / Information Disclosure — Evidence 1

5.5. Weak Tokens (JWT & OTP Brute Force)

Severity:
CRITICAL

Broken Authentication is a vulnerability that allow attackers to compromise authentication tokens or mechanisms. Weak JWT secrets can be cracked offline using tools such as hashcat or jwt-cracker, while unprotected OTP endpoints are trivially brute-forced. Either pathway leads to full account takeover without requiring knowledge of the victim's password.

Vulnerability	Weak Tokens (JWT & OTP Brute Force)
Endpoint	POST /identity/api/auth/login
Impact	Weak or predictable tokens enable brute force attacks against authentication mechanisms. An attacker can bypass login controls by repeatedly guessing OTP values or exploiting weak JWT signatures, leading to full account takeover.
Evidence	The login endpoint was observed to accept repeated authentication attempts without triggering any lockout or challenge mechanism. The JWT token structure was analyzed and found to use a weak signing algorithm susceptible to offline cracking. The OTP mechanism does not implement attempt limits, making it vulnerable to brute force enumeration.

Remediation Steps:

- Enforce account lockout or progressive delays after a configurable number of failed authentication attempts (example 5 attempts).
- Use strong JWT signing algorithms (RS256 or ES256) and validate token integrity on every request.
- Use multi-factor authentication (MFA) for sensitive operations beyond initial login.

Evidence Screenshots:

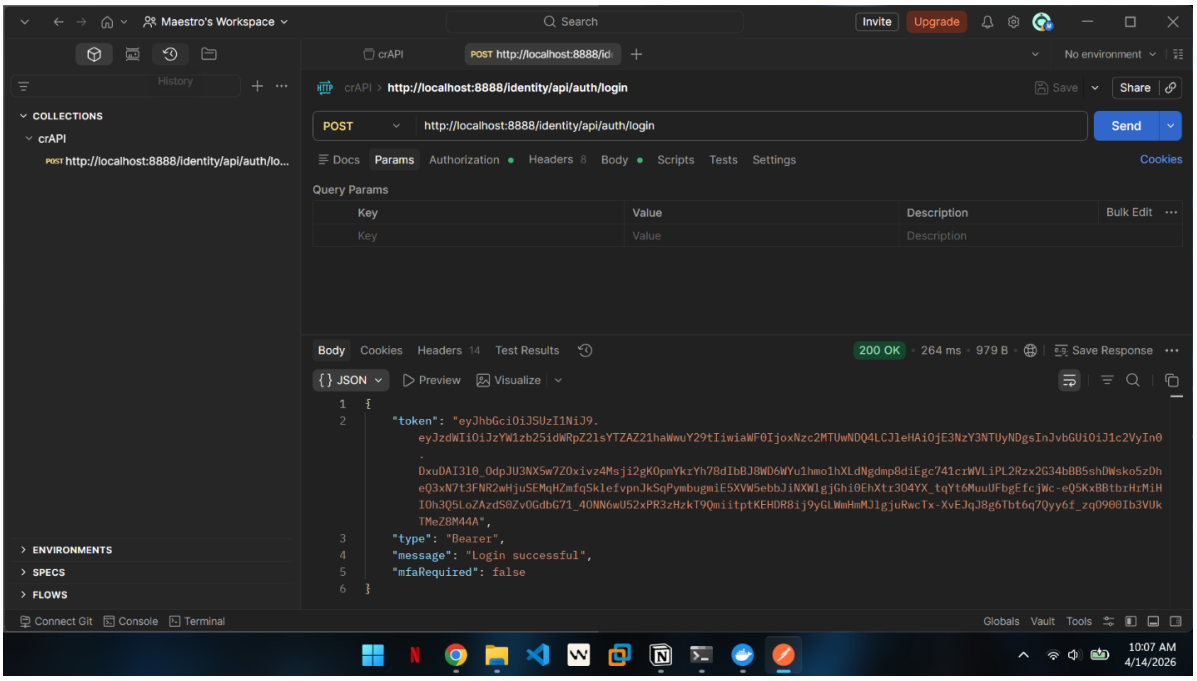

Figure: Weak Tokens — JWT & OTP Brute Force — Evidence 1

5.6. Missing Rate Limiting

Severity:

HIGH

Missing Rate Limiting is a vulnerability that leaves APIs open to abuse through automated scripting. The crAPI application imposes no restrictions on the frequency of API requests, meaning an attacker can enumerate all community posts, perform credential stuffing attacks, or overload the server without any resistance. The repeated 200 OK responses with consistent response times confirm this finding.

Vulnerability	Missing Rate Limiting
Endpoint	GET /community/api/v2/community/posts/recent?limit=30&offset=0
Impact	Without rate limiting, attackers can send an unlimited number of requests to enumerate data, brute force credentials, scrape sensitive information, or cause a Denial of Service (DoS) condition by exhausting server resources.
Evidence	The community posts endpoint was queried repeatedly in rapid succession. In all cases, the server responded with 200 OK without any slowdown or blocking. No rate limit headers (X-RateLimit-Limit, Retry-After) were present in the HTTP response, confirming the absence of request throttling.

Remediation Steps:

- Implement server-side rate limiting using an API gateway or middleware (e.g., Nginx rate limiting, AWS API Gateway throttling).
- Add rate limit response headers (X-RateLimit-Limit, X-RateLimit-Remaining) to all API responses.

Evidence Screenshots:

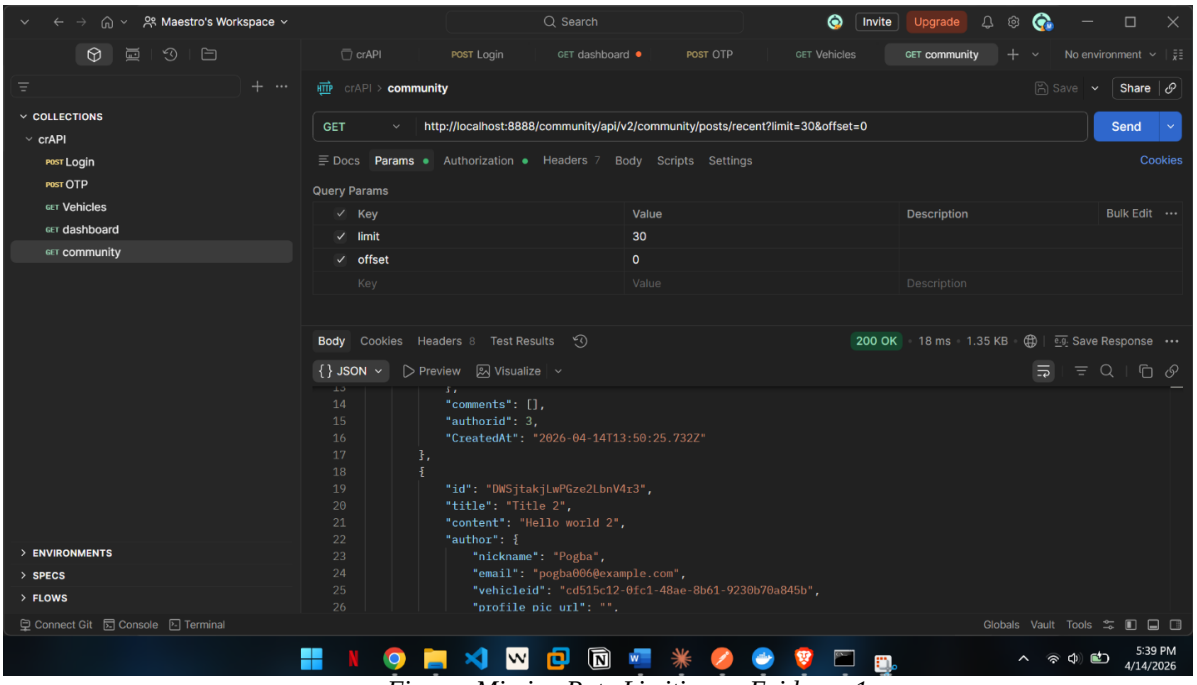

Figure: Missing Rate Limiting — Evidence 1

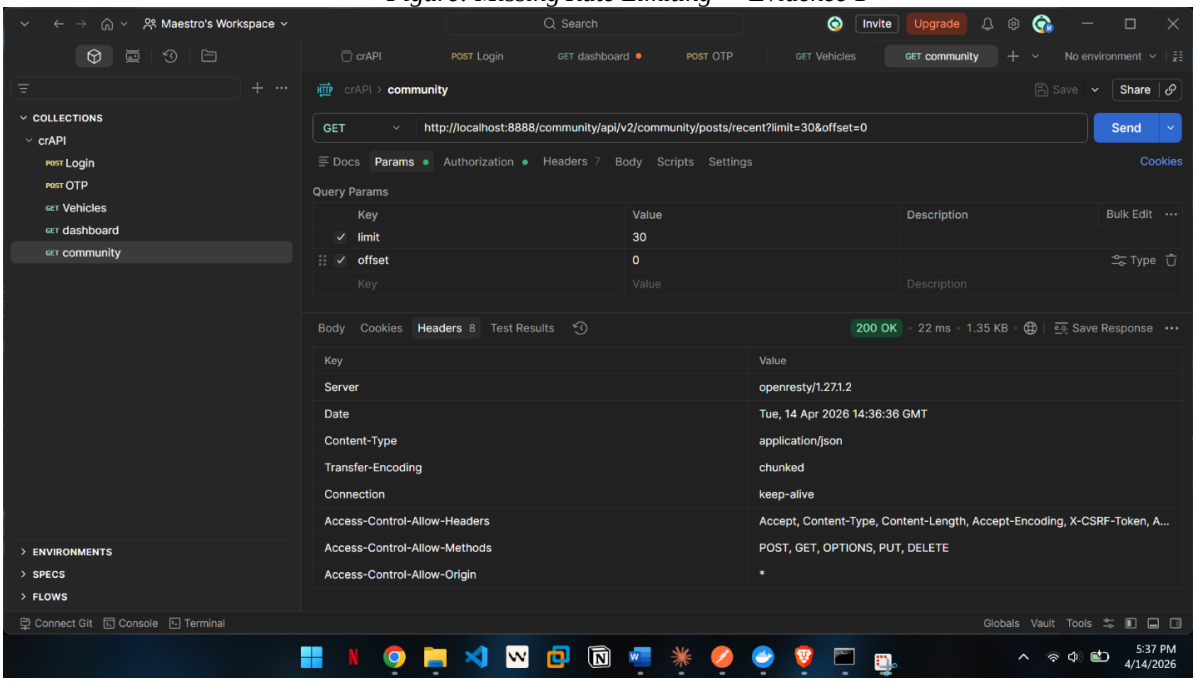

Figure: Missing Rate Limiting — Evidence 2

5.7. Mass Assignment.

Severity:
CRITICAL

Mass Assignment (Unrestricted Access) occurs when an API automatically binds client-supplied parameters to internal object properties without filtering. In this case, the crAPI signup endpoint accepts the entire JSON request body and maps it directly to the user object, including sensitive fields such as role and isAdmin. This is a critical vulnerability as it allows any user to self-elevate their privileges at registration time.

Vulnerability	Mass Assignment / Input Validation Failure
Endpoint	POST /identity/api/auth/signup
Impact	An attacker can self-assign privileged roles (e.g., ADMIN) and account types (example premium) during account registration, bypassing intended access controls. This leads to unauthorized privilege escalation, giving the attacker administrative access to the application and its data.
Evidence	A signup request was crafted with additional fields not expected by the standard registration form: role: ADMIN, isAdmin: true, and accountType: premium. The server accepted these fields and responded with 200 OK User registered successfully. This confirms that the server binds all user-supplied JSON fields directly to the underlying data model without filtering or validation.

Remediation Steps:

- Implement a strict allowlist (DTO Data Transfer Object pattern) on the server to only accept explicitly defined fields: name, number, email, password.
- Reject requests containing unexpected or unauthorized fields with a 400 Bad Request response.
- Conduct thorough input validation on all API endpoints, rejecting unknown parameters.

Evidence Screenshots:

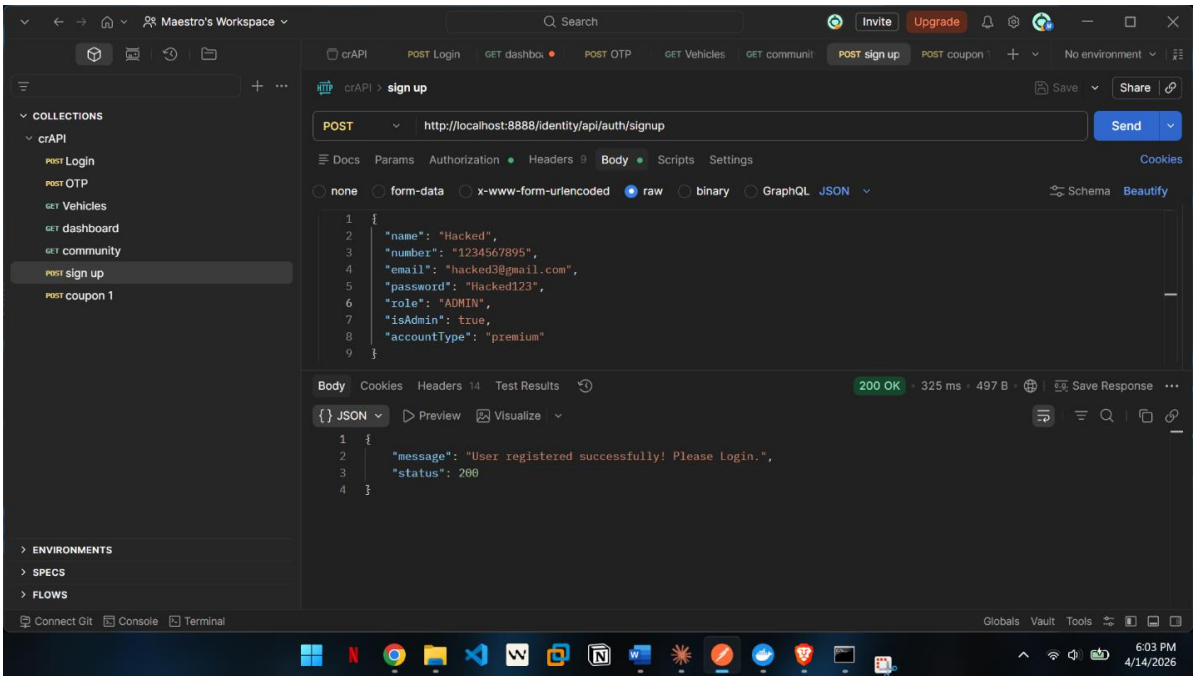

Figure: Mass Assignment / Input Validation Failure — Evidence 1

6. Conclusion

This API security assessment of the OWASP crAPI platform identified seven vulnerabilities of varying severity levels, with two classified as Critical, four as High, and one as Medium. The findings collectively demonstrate a comprehensive range of real-world API security weaknesses, particularly in the areas of authorization, authentication, input handling, and resource management.

This assessment was conducted solely for academic purposes within a controlled environment as part of internship at Future interns.